

AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

AMANDEEIP KAMMARI, [TODO: Institute / Department] (Roll No. 12341060), India

Village ponds are among the cheapest ways to harvest rainwater in rural India, but deciding where to dig one, and how big to make it, normally needs a field survey and a hydrologist. We present a web application that does this preliminary planning from free public data. Given a village name or a map click, the server builds a 20 m elevation model from global terrain tiles (or from an uploaded KML contour map), routes surface flow with Priority-Flood depression filling and the D8 algorithm, and identifies land available for excavation by combining an OpenCV classifier on satellite imagery with OpenStreetMap exclusions and optional government parcels. Candidate sites are scored on flow accumulation, slope, concavity and land suitability, and a ranking algorithm returns up to five sites whose catchments are hydrologically independent. Twenty years of daily CHIRPS rainfall drive a daily SCS Curve Number runoff model, and each pond is sized to half of the 75 %-dependable annual runoff. The system is a modular FastAPI monolith with SQLite, a three-level cache and a fallback chain for every external service. It was validated against independent data (CHIRPS rainfall within 8.9 % of IMD normals; land classifier 92 % accurate, $\kappa = 0.85$), covered by 35 automated tests, load-tested, and deployed on a college lab server, where system testing found and fixed six defects that unit tests had missed. For Hiware Bazar (Maharashtra) it recommends five ponds storing 121 155 m³. A first analysis takes about 44 s, dominated by the rainfall download; a cached one takes 1.3 s, and eight concurrent cached analyses complete in 1.3 s each.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

Source code: <https://github.com/amandeeip01/village-pond-planner>

Video explanation: <https://www.youtube.com/watch?v=StI46Pxp3kM>

1 INTRODUCTION

Most of India's net sown area is rain-fed, and most of its rain falls in the four months of the south-west monsoon. Small surface storages — farm ponds, village tanks and the *Amrit Sarovars* built under national programmes [1] — hold this water back for irrigation, livestock and groundwater recharge. Pond-based harvesting is effective because it is cheap, built with local labour and machinery, and recharges aquifers while storing surface water. A pond is only useful, however, if three early decisions are right:

- (1) **Where?** On a drainage line that actually receives runoff, on gentle ground, and on land the village is allowed to dig — usually panchayat or government land.
- (2) **How much water will arrive?** This depends on catchment area, land cover, soil and the rainfall regime, *including dry years*.
- (3) **How big?** A pond much larger than its dependable inflow rarely fills; one much smaller wastes the catchment. Depth trades evaporation against excavation cost and safety.

The goal of this assignment is a web application that analyses terrain and rainfall to recommend suitable pond locations and to estimate catchment area, rainfall, runoff, pond depth and storage capacity, all shown on an interactive map. Section 2 states the requirements, Section 3 the architecture and design decisions, Section 4 the algorithms, and Section 5 the implementation and deployment. Section 6 maps the design to Computer System Design themes; Section 7 presents the case study, validation, testing and measured performance; Section 8 discusses limitations.

1.1 Motivation

Choosing a pond site by hand is hard for four reasons. *Terrain is subtle*: on the gently sloping Deccan plateau a drainage line can be invisible on the ground, yet a few metres of relief decide where water collects. *Rainfall data is scattered*: IMD station records are sparse, gridded products disagree (we measured one widely used dataset to be 92% too wet in a rain-shadow city), and an administrator rarely knows the dependable dry-year rainfall that should govern design. *Land ownership is not open*: finding land that is both free and hydrologically useful needs local knowledge. *Sizing needs a runoff model*, which is beyond most panchayat staff. An automated tool that turns a village name into a short, quantified list of candidates lets an engineer spend field time verifying a handful of good options instead of searching a whole village.

1.2 Scope of the Project

The system **does**: find a village by name or map click; build a terrain model from an elevation service or an uploaded KML/KMZ contour map; display satellite imagery, contours, drainage and land cover; identify land available for excavation; recommend up to five ranked pond sites with independent catchments (or evaluate a site chosen by the user); delineate each catchment; retrieve 5–40 years of daily rainfall; estimate runoff; recommend pond dimensions, storage, earthwork and water losses; and save, list and export analyses.

The system **does not**: perform structural or geotechnical design of embankments, inlets or spillways; determine legal land ownership (it can only use parcels supplied by the administrator); model groundwater, measured infiltration or water quality; estimate costs; or replace a field survey. It is a *pre-feasibility* tool.

1.3 Contributions

- An end-to-end pipeline from a place name to ranked, dimensioned pond sites using only free public data and no GIS server software.
- A ranking algorithm (Algorithm 1) that guarantees independent alternatives, and a land-availability model that fuses image classification, OpenStreetMap and user parcels.
- A resilient service design — caching, fallbacks, a circuit breaker and retries — driven by load testing and a real deployment on an unreliable campus network.
- A quantitative validation of every data source, which changed the system's default rainfall dataset, and a test suite that includes regression tests for every defect found.

2 PROBLEM STATEMENT AND REQUIREMENTS

Given a location in rural India, the system must recommend where a rainwater-harvesting pond should be excavated and how large it should be, and justify the recommendation with terrain, land, rainfall and runoff evidence on a map. Table 1 maps each functional requirement of the brief to the module that implements it and the output that shows it.

2.1 Non-Functional Requirements

Table 2 lists the non-functional requirements we set, each with a measurable target, and what was measured (Section 7.5).

Table 1. Functional requirements and where they are implemented

Requirement	Implemented in	How	Visible as
Satellite imagery display	geocode.py, static/app.js	Nominatim village search; Esri World Imagery basemap with labels	Satellite map, search box
Contour map visualization	layers.py, kml_parser.py	Contours traced from the DEM with OpenCV, or uploaded KML contours; relief overlay	Contour and relief layers
Available-land identification	landcover.py	OpenCV classification + OpenStreetMap exclusions + parcels + slope + sea mask	Available-land and land-cover layers
Catchment area estimation	hydrology.py	Priority-Flood, D8, accumulation, reverse traversal, boundary tracing	Catchment polygon, area, relief
Historical rainfall query	rainfall.py	CHIRPS (ClimateSERV); fallback Open-Meteo ERA5, then NASA POWER	Rainfall tab, charts
Runoff volume estimation	runoff.py	Daily SCS Curve Number with antecedent moisture; rational-method check	Runoff tab
Pond depth / storage recommendation	pond.py, design.py	Site scoring and ranking; frustum sized to dependable runoff	Pond tab, cross-section, footprint
Combined overlay / results view	static/, pipeline.py	Leaflet layers, Chart.js, candidate table, exports	Map + sidebar, JSON/GeoJSON

Table 2. Non-functional requirements: targets and measured results

Quality	Target	Measured	Status
Performance	First analysis of a new village ≤ 60 s; repeat ≤ 5 s; light endpoints < 50 ms	44.2 s first; 1.3 s repeat; health 2.4 ms median, cached rainfall 8 ms	met
Scalability	≥ 8 concurrent analyses on one instance without degrading results	8 concurrent cached analyses: 1.32 s median, 1.35 s max (after the fixes in Section 7.6)	met
Availability	Failure of any one external service degrades, never aborts, and is reported	Fallback for every source; degradations listed in warnings; verified on a network where four services failed intermittently	met
Usability	Analysis from a village name alone; keyboard and screen-reader use; WCAG AA contrast; current desktop browsers	Only lat/lon required; ARIA tabs, focus, skip link; 5.6:1 text contrast	met
Security	Validate all input; 25 MB upload cap; no secrets in the repository; external calls from the server only	Pydantic validation; size check; full git history scanned before publishing	met
Portability	pip-installable without GIS servers; container image; free hosting	Runs on macOS and Ubuntu 24.04 lab server; Dockerfile and Render blueprint	met
Transparency	Every number traceable to data and method	Response carries sources, methods, parameters, timings, warnings	met

3 SYSTEM ARCHITECTURE AND HIGH-LEVEL DESIGN

The system is a three-tier web application (Figure 1). The *client* is a static single-page application (HTML, CSS, JavaScript with Leaflet and Chart.js). The *application server* is one FastAPI process holding all analysis logic in a pipeline of independent modules. The *persistence tier* is an SQLite database (saved analyses and an API-response cache) plus a disk

cache of map tiles. The *external API layer* consists of seven free public services, which only the server calls; this lets it set proper User-Agent headers, cache responses, and stay within fair-use limits.

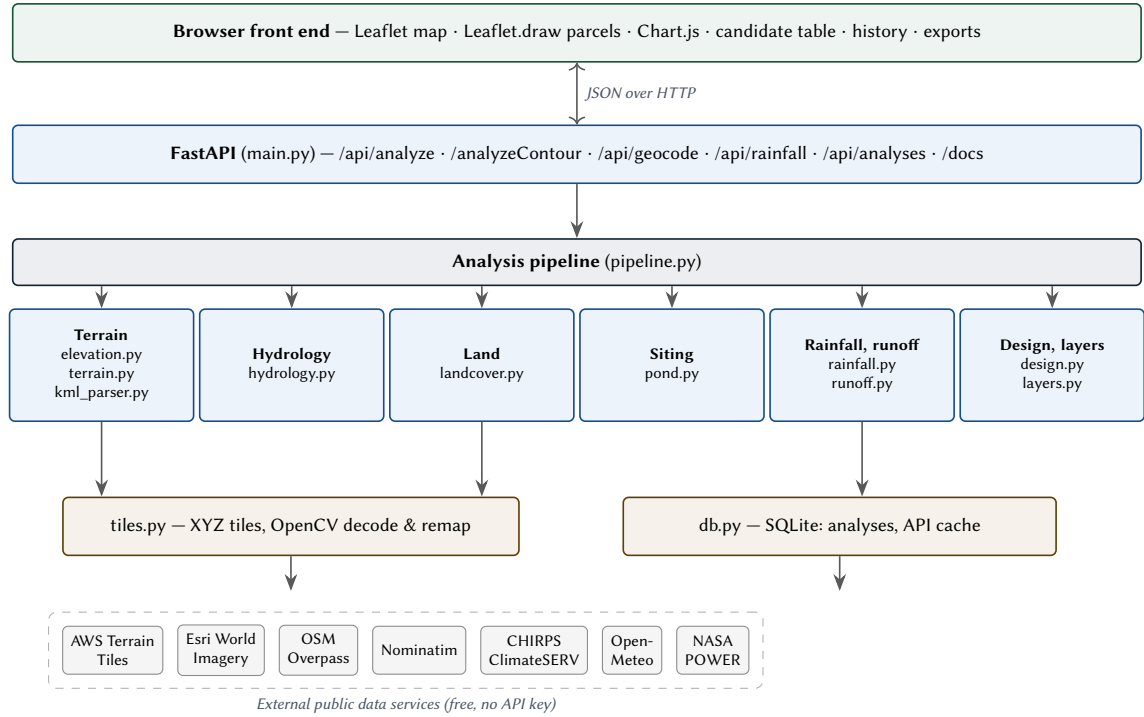


Fig. 1. High-level system architecture. Arrows show the direction of calls. Only the server talks to external services, and their responses are cached.

Figure 2 shows the processing stages of one analysis. The two inputs — a village location or an uploaded contour map — converge on a common in-memory DEM, after which the pipeline is identical, so a new terrain source can be added without touching the hydrology. Figure 3 shows the same request as a sequence over time: the slow rainfall download (two ten-year CHIRPS chunks) runs in parallel with the evapotranspiration download, and on a repeat request every external call is served from a cache.

3.1 Technology Stack

Table 3 lists the stack actually used.

Deviations from the suggested stack. The brief suggests IMD for rainfall. IMD gridded rainfall is distributed as bulk files rather than through a query API, so CHIRPS was chosen after validating three datasets against IMD station normals: NASA POWER was off by up to +92 % in the rain shadow, CHIRPS by 8.9 % on average. We did not use GDAL or a GIS server: the required algorithms are small enough to implement directly in NumPy and OpenCV, which keeps the system installable from pip wheels on free hosting. SQLite replaces a client-server database because the data is a small document store with one writer process.

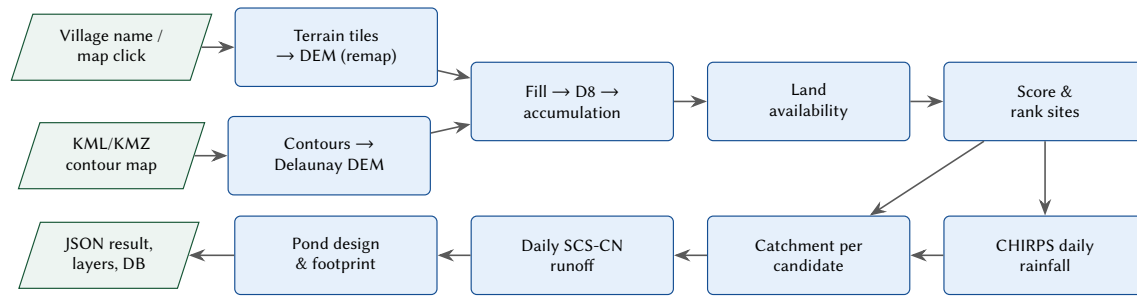


Fig. 2. Processing stages of one analysis. Rainfall is fetched once, at the top site, and shared by all candidates.

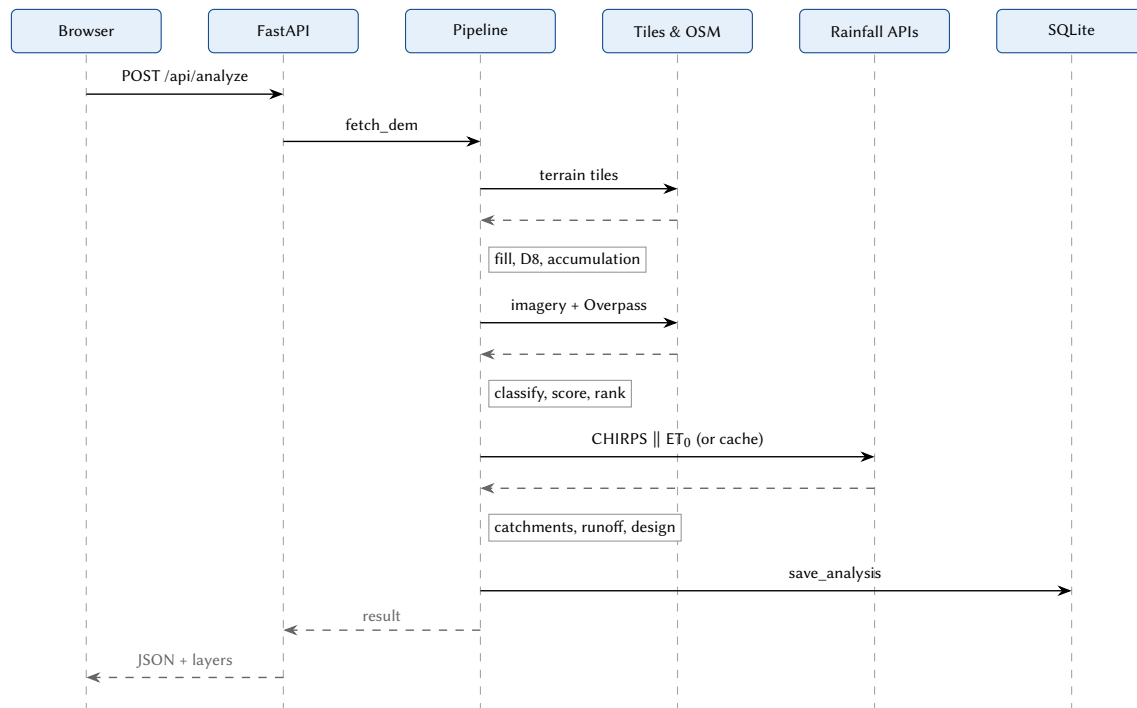


Fig. 3. Sequence of one village analysis request.

3.2 Key Design Decisions

Table 4 records the main design choices with the alternatives we considered.

Table 3. Technology stack

Layer	Technology	Why
Language, server	Python 3.12+, FastAPI 0.115, Uvicorn 0.34, Pydantic 2.10	Typed request/response models give validation and OpenAPI docs for free; scientific Python ecosystem
Numerics, imaging	NumPy 2.2, SciPy 1.15, OpenCV 5 (headless)	Vectorised grids; Delaunay interpolation; tile decoding, remap, contours, morphology, rasterisation
Database	SQLite (standard library)	Small document store, single writer, zero administration
Frontend	HTML/CSS/JS, Leaflet 1.9, Leaflet.draw 1.0, Chart.js 4.4	No build step; small; the brief asks for a basic library
Terrain data	AWS Terrain Tiles (SRTM/Copernicus ~30 m); fallback Open-Meteo elevation	Global, key-less, one request per 256×256 tile, cacheable
Imagery, map data	Esri World Imagery; OpenStreetMap Overpass and Nominatim	Free high-resolution imagery; buildings, roads, water, land use; place search
Rainfall, ET ₀	CHIRPS v2.0 via ClimateSERV; fallback Open-Meteo ERA5, NASA POWER	Most accurate against IMD normals (Table 14)
Testing	pytest 9, FastAPI TestClient, curl-based end-to-end and load scripts	Unit, API, system and load levels
Deployment	Docker, Render blueprint; venv + Uvicorn on the lab server	Reproducible, free; works without Docker where unavailable

Table 4. Design decisions and rationale

Decision	Alternatives	Rationale
FastAPI	Flask, Django	Pydantic models validate requests and generate the OpenAPI reference; async-capable; same schemas document and validate.
NumPy/SciPy/OpenCV hydrology	GDAL, WhiteboxTools, GRASS	Installs from pip on any platform and on free hosting; every algorithm is visible and testable.
Terrain tiles for the DEM	OpenTopography, point APIs	One request per tile; a point API would need 40 000 queries per 200×200 grid.
Local equirectangular projection	UTM via pyproj	Distortion far below DEM error over a few km; avoids a PROJ dependency.
CHIRPS default rainfall	ERA5, NASA POWER, IMD grids	Best agreement with IMD normals; IMD grids are files, not an API.
Daily SCS-CN	Rational method, monthly balance	Captures storm-size non-linearity and antecedent moisture; standard in Indian watershed manuals.
Rule-based OpenCV + OSM for land	Deep segmentation model	No labelled Indian village data; explainable; no GPU; 92 % accurate.
Modular monolith	Microservices	One developer; stages exchange large in-memory arrays; one deployable unit.
SQLite	PostgreSQL, MongoDB	Single writer, small data; isolated behind db.py so it can be swapped.

4 METHODOLOGY

4.1 Terrain and Elevation Analysis

Grid. For a village centre (λ_0, φ_0) and radius R (default 2 km) the server lays out an $N \times N$ grid (default $N = 200$, cell size $\Delta = 2R/N = 20$ m) in a local equirectangular projection,

$$x = R_E \cos \varphi_0 (\lambda - \lambda_0) \frac{\pi}{180}, \quad y = R_E (\varphi - \varphi_0) \frac{\pi}{180}, \quad R_E = 6\,371\,008.8 \text{ m}. \quad (1)$$

Elevation. AWS Terrain Tiles encode elevation in the RGB bytes of 256×256 Web-Mercator PNG tiles:

$$z = 256R + G + B/256 - 32768 \text{ [m]}. \quad (2)$$

The tiles covering the grid are downloaded in parallel (8 threads), decoded with OpenCV, stitched into a mosaic and sampled at every cell centre with `cv2.remap` (bilinear). Decoding happens *before* resampling, because interpolating the packed bytes would mix the high and low bytes of neighbouring pixels and produce errors of hundreds of metres. If the tile service fails, a 40×40 lattice is queried from the Open-Meteo elevation API and interpolated bicubically.

Contour-map input. For an uploaded KML/KMZ map, each contour's elevation is detected from coordinate z values, ExtendedData, the placemark name or the folder name (the first strategy covering 90 % of lines with varying values wins). Vertices are interpolated linearly over their Delaunay triangulation, with nearest-neighbour fill outside the convex hull.

Derived terrain. Contours for display are traced from the DEM with `cv2.findContours` at an automatic interval with major and minor lines; slope and plan curvature come from finite differences; a colour-relief hillshade is rendered as a PNG overlay.

Sea mask. Terrain tiles report the sea as a flat 0 m surface with bathymetry further out. A connected low region ($z \leq 0.05$ m) is marked as sea only if it contains at least 0.25 % of the grid's cells as flat sea surface ($|z| \leq 0.05$ m) or bathymetry ($z < -5$ m). This excludes the sea and the shallow seabed joined to it, while keeping inland land below sea level, such as the polders of Kuttanad, Kerala. An area that is mostly sea is rejected.

4.2 Catchment Area Delineation

Catchments are derived with standard raster flow routing (Table 5).

Table 5. Hydrology algorithms (n = number of cells, 40 000 by default)

Step	Method	Complexity
Depression filling	Priority-Flood [2]: seed a min-heap with edge cells; pop the lowest; raise each unvisited neighbour to at least its level + 10^{-4} m	$O(n \log n)$
Flow direction	D8 [3]: steepest distance-weighted drop $s_k = (z_c - z_k)/d_k$, $d_k \in \{1, \sqrt{2}\}\Delta$, as eight vectorised array shifts	$O(n)$
Flow accumulation	Visit cells in decreasing filled elevation, adding each count to its receiver	$O(n \log n)$ (sort)
Catchment	Breadth-first traversal of the reversed D8 graph from the outlet	$O(n)$
Boundary	Collect cell edges between inside and outside, stitch into a ring, drop collinear vertices; area = cells $\times \Delta^2$	$O(n)$
Edge inflow	Border cells draining inward flag all cells downstream of them	$O(n \log n)$

Priority-Flood was chosen over iterative pit filling, which can be $O(n^2)$; the whole terrain stage takes about 60 ms. In village mode, cells whose catchment may extend beyond the map edge are excluded from siting, so no reported catchment is truncated.

Available land. Esri imagery is resampled to 5 m and classified per pixel: *vegetation* where the Green–Red Vegetation Index $GRVI = (G - R)/(G + R)$ exceeds an Otsu threshold [4] clamped to $[0, 0.10]$; *built-up* where bright, highly textured pixels (local s.d. over 15 m above $\max(18, P_{90})$) reach a density of 0.30 over 40 m; *water* for dark, blue-dominant, smooth pixels; *open land* otherwise. Morphological opening and closing remove speckle. OpenStreetMap buildings

Algorithm 1 Ranking independent candidate sites

Require: score grid S , D8 directions D , number n , minimum separation $d_{\min}$

```

1:  $\mathcal{A} \leftarrow []$ ;  $\mathcal{M} \leftarrow []$  ▷ accepted sites, their catchment masks
2: for all cells  $c$  in decreasing order of  $S$  do
3:   if  $|\mathcal{A}| = n$  then break
4:   if  $\exists a \in \mathcal{A} : \|c - a\| < d_{\min}$  then continue ▷ too close
5:   if  $\exists m \in \mathcal{M} : m[c]$  then continue ▷  $c$  upstream of an accepted site
6:    $m_c \leftarrow \text{CATCHMENT}(D, c)$ 
7:   if  $\exists a \in \mathcal{A} : m_c[a]$  then continue ▷  $c$  downstream of an accepted site
8:   append  $c$  to  $\mathcal{A}$  and  $m_c$  to  $\mathcal{M}$ 
9: end for
10: return  $\mathcal{A}$ 

```

(+20 m), roads (+4–30 m by class), railways, rivers, water bodies, forest, settlements, institutions and protected areas are excluded; grassland, scrub, village green or public-ownership tags are *preferred*; farmland is allowed at lower priority. Administrator parcels, when given, restrict siting to them, and ground steeper than 8° is excluded.

Site scoring and ranking. Candidates are available cells on the drainage network (top 8 % of accumulation, relaxed to the 85th, 75th and 60th percentiles if available land does not reach it). Each candidate c is scored

$$S_c = 0.75(0.45 \hat{A}_c + 0.35(1 - \hat{\theta}_c) + 0.20 \hat{\kappa}_c) + 0.25 L_c, \quad (3)$$

with $\hat{A}$ the min–max normalised $\log(1 + \text{accumulation})$, $\hat{\theta}$ normalised slope, $\hat{\kappa}$ normalised concavity and L the land score. Taking the top five scores would return five adjacent cells on one stream, so Algorithm 1 enforces spatial separation and hydrological independence: no site lies upstream or downstream of another, so their inflows can be summed without double counting. A user-chosen location is instead snapped to the highest-accumulation cell within 60 m and becomes rank 1.

Assumptions and limitations. D8 sends all flow to one neighbour and cannot model divergent flow; the ~ 30 m source DEM cannot see bunds or drains narrower than about 30 m; land availability is inferred, not legal ownership.

4.3 Rainfall Data Integration

Daily precipitation comes from CHIRPS v2.0 [5] (0.05° , satellite–gauge blend) through the ClimateSERV API, downloaded as two ten-year chunks in parallel. If CHIRPS fails, the system falls back to Open-Meteo’s ERA5 reanalysis [6], then to NASA POWER; transient network errors are retried with back-off, definitive ones (e.g. HTTP 429) move straight to the next source. Reference evapotranspiration comes from Open-Meteo (FAO-56 Penman–Monteith) or, from NASA POWER temperatures, by Hargreaves–Samani [7]:

$$ET_0 = 0.0023 (0.408 R_a) (T_{\text{mean}} + 17.8) \sqrt{T_{\text{max}} - T_{\text{min}}}. \quad (4)$$

The default record is the last 20 complete years (2006–2025), configurable from 5 to 40. If the administrator knows the local IMD normal $\bar{P}^*$, the daily series is scaled by $\bar{P}^*/\bar{P}$, preserving the storm pattern while removing bias.

From the daily series the system computes the mean, median, standard deviation and coefficient of variation of annual totals; the 75 %-dependable rainfall (25th percentile of annual totals, equalled or exceeded three years in four); the driest and wettest years; rainy days (≥ 2.5 mm, the IMD definition); the heaviest day; the June–September share; the monthly climatology; and annual and dry-season (October–May) ET_0 .

4.4 Runoff Estimation

Runoff is simulated for every day of the record with the SCS Curve Number method [8]:

$$S = \frac{25400}{CN} - 254, \quad I_a = 0.2S, \quad Q = \begin{cases} \frac{(P - I_a)^2}{P - I_a + S}, & P > I_a \\ 0, & \text{otherwise} \end{cases} \quad [\text{mm}]. \quad (5)$$

Curve numbers (Table 6) come from each land-cover class for the chosen soil group. Each day's CN is adjusted to antecedent moisture from the previous five days' rain P_5 (AMCI if $P_5 < 35.6$ mm, AMC III if $P_5 > 53.3$ mm) with Hawkins' conversions [9],

$$CN_I = \frac{CN}{2.281 - 0.01281 CN}, \quad CN_{III} = \frac{CN}{0.427 + 0.00573 CN}. \quad (6)$$

Runoff is computed per class and area-weighted — more accurate than one composite CN because the equation is non-linear. Annual volume is $V = Q \cdot A$; the mean and the 75 %-dependable volumes are reported, with the rational method $V = A\bar{P}C$ as a cross-check.

Table 6. Curve numbers (AMC II) by land cover and hydrologic soil group, after TR-55

Class	TR-55 cover type	A	B	C	D	Rational C
Water	rain on water surface	98	98	98	98	1.00
Built-up	residential, $\approx 85\%$ impervious	89	92	94	95	0.75
Vegetation	row crops, straight row, good	67	78	85	89	0.25
Open land	pasture/fallow, poor	68	79	86	89	0.35
Forest (OSM)	woods, fair	36	60	73	79	0.15

4.5 Pond Design

The pond is a rectangular excavated frustum (Figure 4) with side slope $z:1$ (default 1.5), length $1.5 \times$ width, long side along the contour, and 0.5 m freeboard. For top width W , length L and depth d the prismoidal formula gives the exact volume

$$V = \frac{d}{6} (LW + 4(L - zd)(W - zd) + (L - 2zd)(W - 2zd)), \quad (7)$$

and W is found for a target V by bisection. The target is half of the dependable annual runoff, so the pond fills about twice even in a dry year. Depth starts at 3 m (MGNREGA and Amrit Sarovar practice [1]) and grows in 0.5 m steps to at most 4.5 m if the footprint exceeds 80 % of the connected patch of available land or the 2 ha village-pond cap; if it still does not fit, the volume is reduced and the response says so — or, if not even the smallest pond fits, reports that no pond fits at that site. Small ponds are made shallower so the bed stays at least 4 m wide. The design also reports excavation volume, dry-season evaporation ($1.05 \text{ ET}_{0,\text{Oct-May}}$ over the half-depth water area [7]), seepage by soil group, and fills per dependable year.

5 IMPLEMENTATION

The back end is 3 515 lines of Python in sixteen modules (Table 7); the front end is 1 204 lines of HTML, CSS and JavaScript; the tests are 374 lines. `main.py` handles only HTTP; `pipeline.py` orchestrates; each analysis module is a function of arrays and parameters that can be tested in isolation; network and disk I/O are confined to `tiles.py`, `rainfall.py`, `landcover.py`, `geocode.py` and `db.py`.

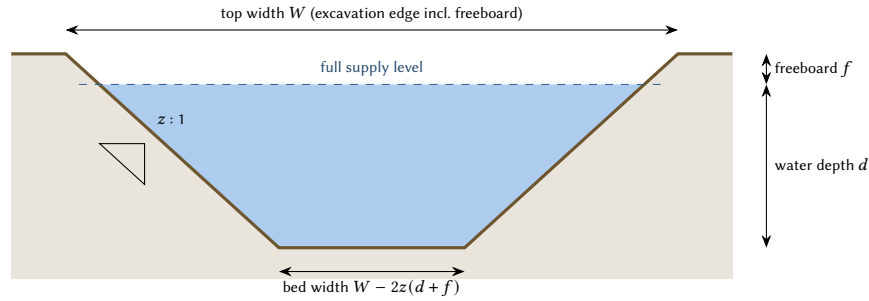


Fig. 4. Cross-section of the excavated pond with the design variables.

Table 7. Back-end modules

Module	Lines	Responsibility
main.py	243	Routes, validation, error mapping, static front end
schemas.py	98	Pydantic request/response models (drive /docs)
pipeline.py	365	Orchestration, sea mask, per-candidate evaluation, persistence
tiles.py	169	Web-Mercator maths, parallel tile download with retries, disk cache, mosaic
elevation.py	91	DEM from terrain tiles, Open-Meteo fallback
terrain.py	197	Projection, grid, contour interpolation, slope, curvature
kml_parser.py	225	KML/KMZ parsing, four elevation-detection strategies
hydrology.py	239	Priority-Flood, D8, accumulation, edge inflow, catchment, boundary
landcover.py	501	OpenCV classification, Overpass client (cache, circuit breaker), rasterisation, tiers
pond.py	263	Site scoring, independent ranking, snapping, natural-basin storage
rainfall.py	340	CHIRPS/Open-Meteo/NASA clients, parallel fetch, cache, scaling, statistics
runoff.py	150	Curve numbers, AMC adjustment, daily SCS-CN, rational method
design.py	192	Frustum geometry, depth selection, losses, footprint
layers.py	262	Contours, drainage network, land polygons, PNG overlays
db.py, geocode.py	180	SQLite persistence and cache; Nominatim client

5.1 Backend and API Design

The API is REST over JSON (Table 8); the full schema is generated from the Pydantic models and served at /docs. One analysis is a single synchronous request that returns one self-describing document (Listing 1), so the client needs no follow-up calls and a saved result can be re-displayed exactly.

```
POST /api/analyze {"lat": 19.068, "lon": 74.601, "village": "Hiware Bazar"}

{
  "status": "success", "mode": "village", "analysis_id": 22,
  "pond_site": {
    "location": {"lat": 19.07995, "lon": 74.59600, "elevation_m": 708.95},
    "slope_deg": 0.4, "land_tier": "open_land", "reason": "Rank 1: combined score 0.92 ..."
  },
  "catchment": {
    "area_hectares": 137.6, "perimeter_m": 7720, "relief_m": 44.8,
    "boundary_geojson": {
      "type": "Feature", "geometry": {
        "type": "Polygon", ...
      }
    }
  },
  "rainfall": {
    "source": "CHIRPS v2.0 daily, 0.05 deg", "period": "2006-2025",
    "mean_annual_mm": 712.7, "dependable_75pct_mm": 599, "annual_series": [...]
  },
  "runoff": {
    "composite_curve_number": 85.8, "mean_annual_runoff_m3": 134368,
    "dependable_annual_runoff_m3": 73712
  },
  "pond_design": {
    "storage_capacity_m3": 36856, "water_depth_m": 3.0, "top_length_m": 141.3,
    "top_width_m": 94.2, "excavation_volume_m3": 43604, "notes": [...]
  },
  "candidates": [
    { "rank": 1, ... }, { "rank": 5, ... }
  ],
  "layers": {
    "contours": {...}, "drainage": {...}, "suitable_land": {...},
    "elevation_png": "data:image/png;base64,..."
  },
  "bounds": [...]
}
```

Table 8. HTTP endpoints

Method	Path	Purpose
POST	/api/analyze	Full analysis of a village location (JSON; only lat, lon required)
POST	/analyzeContour	Full analysis of an uploaded KML/KMZ (multipart); alias /findCatchment
GET	/api/geocode?q=	Village search
GET	/api/reverse?lat=&lon=	Place name for a map click
GET	/api/rainfall?lat=&lon=&years=	Rainfall statistics only
GET	/api/analyses	List saved analyses (summaries)
GET / DELETE	/api/analyses/{id}	Retrieve or delete one analysis
GET	/health	Liveness probe for the hosting platform
GET	/, /docs	Front end; interactive API documentation

```
"warnings": [], "method": {"timings_s": {...}, ...}, "processing_seconds": 38.1 }
```

Listing 1. Request and abridged response of POST /api/analyze (Hiware Bazar).

Authentication. There is none: the system is a single-tenant planning tool over public data, so every user may run and view analyses (Section 6 discusses what adding it would involve).

Error handling. All errors share one JSON shape, {"status": "error", "detail": "..."} . Table 9 lists how each failure is handled. The design rule is: a *required* input that cannot be obtained is an error with a precise message; *optional* evidence that cannot be obtained degrades the result and is listed in the response’s warnings.

Table 9. Failure handling

Failure	Behaviour
Invalid input; empty or oversized upload	422 with field-level message; 400; 413
Location mostly at sea	422 “Choose a location on land”
Terrain tile download fails	5 attempts with back-off, then the Open-Meteo elevation fallback; 502 if both fail
Esri imagery missing or placeholder tiles	Land assessment continues on OSM, parcels and slope; warning
Overpass down, rate-limited or firewalled	5 s connect timeout; serialised calls; 7-day cache; 2-minute fail-fast cool-down; warning
CHIRPS unavailable	Retries, then Open-Meteo ERA5, then NASA POWER; failures listed in fallback_errors
All rainfall sources fail	Rational method on a fallback annual rainfall; warning
No pond fits on the land patch	Pond reported as 0 m ³ with an explanatory note (no crash)

Security. Every request is validated by Pydantic (coordinate ranges, enumerations such as soil group A–D, numeric bounds), uploads are capped at 25 MB, and only the server calls third-party services. No secret is needed to run the system; the optional MapTiler key lives in an untracked .env file. Before the repository was made public, its full git history was scanned for credential files, keys and tokens. CORS is open (*) because the API serves only public data; a multi-tenant deployment would restrict it.

5.2 Frontend and Visualization

The front end (Figure 5) is a single page with *Village*, *Contour file* and *History* tabs over a Leaflet map. The Village tab offers search, a preview of the analysis square, parcel drawing and GeoJSON upload with Leaflet.draw, an optional manual pond pin and advanced parameters (soil group, rainfall dataset and normal, storage fraction, depth and slope limits). A staged progress overlay covers the map while an analysis runs. Results appear as a headline storage card with four key figures; a ranked candidate table mirrored by numbered map markers (selecting one redraws everything for it); and *Pond*, *Rainfall*, *Runoff* and *Site & land* sub-tabs with a pond cross-section and Chart.js charts (Figure 6). Layers — contours, drainage, catchment, available land, land-cover and relief rasters, pond footprints — can be toggled. Results export as JSON or GeoJSON and print through a dedicated stylesheet. Accessibility work includes ARIA tab semantics, keyboard operation of lists and table rows, visible focus outlines, a skip link, labelled map and chart regions, reduced-motion support and AA text contrast.

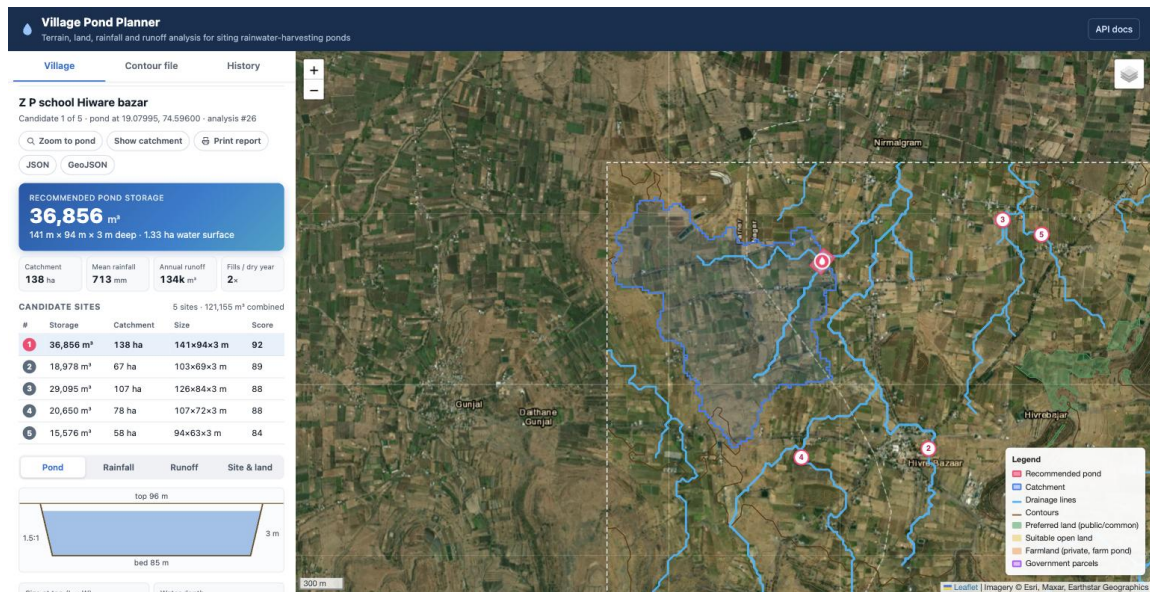


Fig. 5. Application interface showing the results overlay for Hiware Bazar.

5.3 Database and Storage

Figure 7 shows the schema. analyses stores every completed analysis twice: a compact summary for the history list and the full response for exact re-display, with the village, mode, top-site coordinates and a UTC timestamp; an index on created_at serves the newest-first history query, and single analyses are fetched by primary key. api_cache memoises external responses with a time-to-live (Table 10). Map tiles are cached as files keyed by source, zoom and tile coordinates. Spatial results (catchments, footprints, land polygons) are stored as GeoJSON inside the result document rather than in spatial tables, because they are only ever read back whole. All access goes through db.py, which serialises writes with a lock.

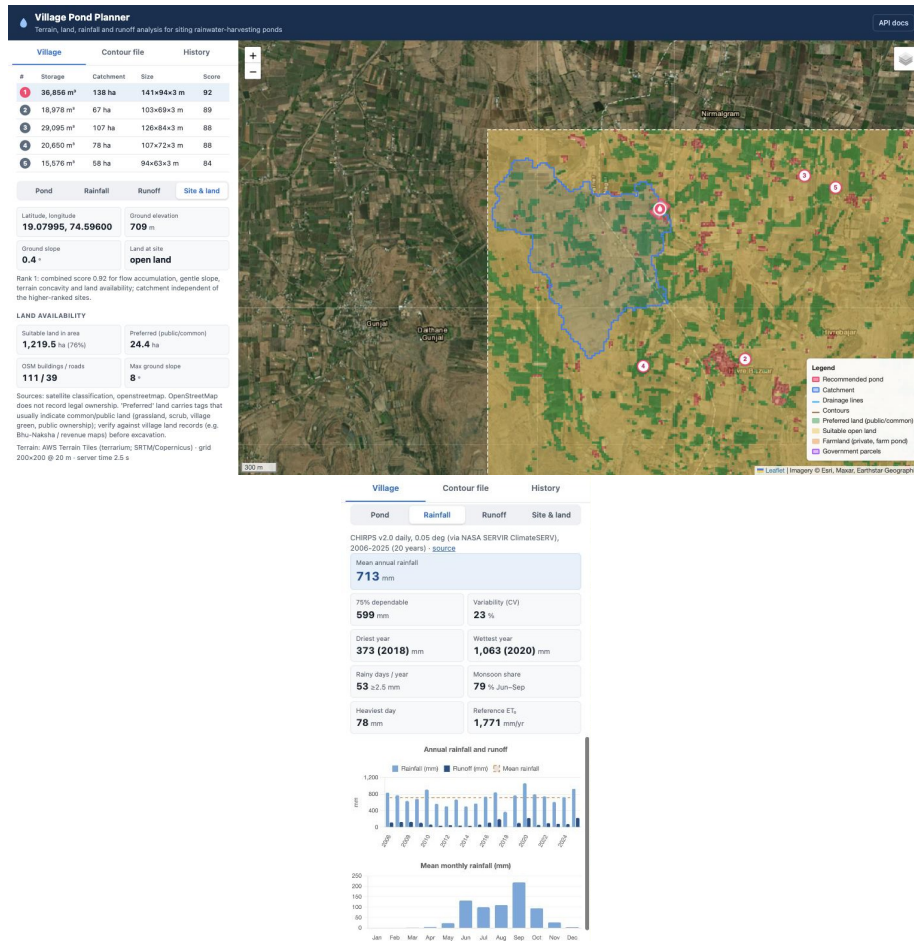


Fig. 6. Further views: (left) land-cover classification overlay — green vegetation, yellow open land, red built-up — with candidate 1's catchment; (right) the Rainfall tab.

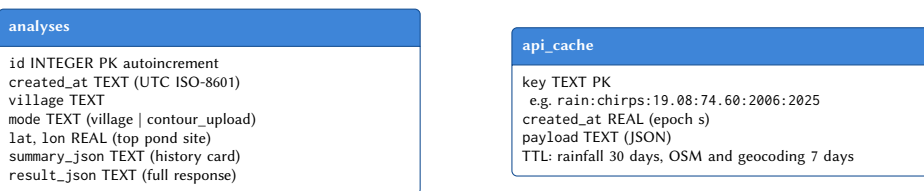


Fig. 7. SQLite schema. An index on analyses(created_at) serves the history list.

5.4 Deployment

The application ships with a Dockerfile (python:3.12-slim) and a Render blueprint with a health check. It was also deployed on a college lab machine, which became an important test environment. Four Ubuntu 24.04 machines (120 cores, 125 GB RAM each) were reachable only through forwarded SSH ports; none had Docker, tmux or rsync, and outbound

Table 10. Cache levels

What	Key	TTL	Effect
Terrain and imagery tiles	cache/<source>/<z>/<x>/<y> (files)	none	No re-download of a village's tiles
Rainfall series	rain:<source>:<lat>:<lon>:<y0>:<y1> (0.01°)	30 d	First run 44 s → repeat 1.3 s
OpenStreetMap features	osm:<bbox to 5 decimals>	7 d	Concurrent users stay within Overpass's limit
Geocoding	geo:<country>:<limit>:<query>	7 d	Respects Nominatim's 1 request/s policy
Results	analyses table	kept	History, re-display, export

internet access was unreliable. We inspected all four and deployed on one, chosen because it reached the most data services: one instance is enough, since the application is CPU-light and limited by external APIs (Section 7.6), and SQLite favours a single writer. The project was copied with tar over SSH, installed into a virtual environment, and started with `nohup setsid uvicorn --host 0.0.0.0` through a small control script (`pond.sh start|stop|restart|status|logs`) so it survives logout. Users reach it through an SSH tunnel (`ssh -L 8080:localhost:8000 sys4`). Measurements on that network are reported in Section 7.4.

6 CSD THEMES AND TOPICS APPLIED IN THE PROJECT

Table 11 maps the Computer System Design themes we applied to concrete parts of the system. Themes we did not implement are discussed after the table.

Themes not implemented. **Load balancing:** one Uvicorn process serves the expected load; the bottleneck is external APIs, not CPU, and SQLite favours a single writer. Scaling out would mean several Uvicorn workers behind a reverse proxy such as Nginx, with the database and caches moved to PostgreSQL and Redis so that instances share them. **Authentication and authorisation:** all data is public and the tool is single-tenant; a multi-panchayat deployment would need login (e.g. OAuth) and per-village ownership of saved analyses. **CI/CD:** there is no pipeline yet; running the 35 tests in GitHub Actions on every push is the obvious next step and would have caught the broken file described in Section 7.4 automatically.

Deep dive 1: caching and resilience around external APIs. Every slow or fragile dependency sits behind a cache and a fallback. Load testing showed why both are needed (Section 7.6): with four concurrent analyses, uncached OpenStreetMap queries exceeded Overpass's per-IP limit, each request waited about 44 s and then silently lost all its OSM exclusions. Adding a cache and a lock alone made things worse when Overpass was down — requests queued behind one another's timeouts, up to 293 s. Only the third step, a circuit breaker that fails fast for two minutes after a failure, gave the intended behaviour: one timeout per outage instead of one per request.

Deep dive 2: a modular monolith instead of microservices. The stages pass large NumPy arrays between them, the team is one person, and the hosting is a single instance. A monolith with strict module boundaries — pure functions of arrays, I/O confined to five modules — gives the testability of services without their operational cost: one process to deploy, monitor and restart, and no network serialisation between stages.

Deep dive 3: choosing data by measurement. The rainfall source was initially Open-Meteo with NASA POWER as fallback. Validation against IMD normals showed errors up to 92 % in exactly the rain-shadow areas where ponds

Table 11. Mapping of core CSD themes/topics to their use in this project

CSD Theme/Topic	Where used in the project	Justification / design rationale
API Design (REST)	<code>/api/analyze</code> , <code>/analyzeContour</code> , <code>/api/rainfall</code> , <code>/api/analyses/{id}</code> ; Pydantic schemas; OpenAPI at <code>/docs</code>	Resources map naturally to REST. One request returns one self-describing document (results, layers, warnings, methods, timings), so the client needs no follow-up calls. The schema is the contract and is validated on every request; errors share one shape.
Caching	Tile files; <code>api_cache</code> (rainfall 30 d, OSM 7 d, geocoding 7 d); saved results (Table 10)	External APIs dominate latency and are rate-limited: a cached analysis takes 1.3 s instead of 44 s. The OSM cache was added after load testing showed concurrent users exhausting the Overpass quota.
Database Indexing / Query Optimization	Primary keys on <code>analyses.id</code> , <code>api_cache.key</code> ; index on <code>analyses(created_at)</code> ; summary stored apart from the full result	The two hot queries — newest n analyses, cache lookup by key — are index scans. The history list reads only the small summary, not multi-megabyte results. No spatial index is needed: no query searches by location.
Concurrency / Asynchronous Processing	Thread pools: 8 parallel tile downloads; two CHIRPS decades in parallel; rainfall and ET_0 in parallel; FastAPI's worker pool per request; database write lock; Overpass lock	Work is I/O-bound on slow APIs, so overlapping downloads cuts first-run time. NumPy and OpenCV release the GIL, so 8 concurrent analyses run as fast as one. Serialising Overpass calls respects its per-IP limit.
Microservices vs. Monolith	Modular monolith: one FastAPI process, layered modules	One developer, one deployable unit, and stages that exchange 40 000-cell arrays — services would add serialisation and network hops for no gain. Module boundaries keep a later split (e.g. a rainfall service) possible.
Design Patterns	Pipeline (<code>pipeline.run</code>); fallback chain of providers (rainfall, elevation); circuit breaker (Overpass cool-down); retry with back-off (tiles, rainfall); data-access module (<code>db.py</code>); DTOs (Pydantic)	Each stage is replaceable; the fallback chain implements graceful degradation uniformly; the circuit breaker stops one failing service from queueing every request behind a timeout; SQL isolated in one module could move to PostgreSQL.
Containerization / Deployment	<code>Dockerfile</code> , Render blueprint with health check; lab-server deployment with <code>venv</code> , <code>nohup</code> <code>setuid</code> , control script, SSH tunnel	Reproducible environment for grading; the health check lets a platform restart a failed instance; the lab deployment works where Docker is not available.
Error Handling and Resilience	Fallbacks for every source; retries; timeouts; warnings; uniform error JSON; input validation; upload cap (Table 9)	Public APIs are rate-limited and sometimes unreachable; the system must degrade, not fail, and say exactly what was degraded.
Algorithms and Complexity	Priority-Flood $O(n \log n)$; vectorised D8 $O(n)$; sorted accumulation; BFS catchment $O(n)$; independent best-first ranking (Table 5)	Terrain analysis of 40 000 cells takes 60 ms, negligible next to I/O. Priority-Flood avoids the $O(n^2)$ worst case of iterative pit filling.
Testing Strategy	30 unit + 5 API tests; validation against independent data; end-to-end, load and deployment tests (Section 7.3)	Unit tests pin formulas to analytic answers; validation measures real accuracy; system tests found six defects that unit tests missed, each now covered by a regression test where possible.
Version Control	Git; public GitHub repository; history scanned for secrets before publishing	Every change is a reviewable commit with a descriptive message; the public link lets graders inspect the code.

matter most, so CHIRPS became the default and the others fallbacks — an architectural decision driven by data quality rather than convenience.

7 RESULTS AND EVALUATION

7.1 Case Study: Hiware Bazar

Hiware Bazar (Ahilyanagar district, Maharashtra) is a model watershed village on the semi-arid Deccan plateau. We analysed a 4 km×4 km area centred at 19.068°N, 74.601°E with soil group C and default parameters. The figures are from a run on 27 September 2026; because imagery, OpenStreetMap and CHIRPS are live sources, later runs can differ slightly (a rerun on 2 October, after one road was removed from OpenStreetMap, swapped the two nearly equal leading sites).

The DEM spans 692–875 m. OpenStreetMap supplied 111 buildings, 39 roads and 23 water features; after all exclusions 76 % of the area (1 219 ha) is suitable for excavation, of which 24.4 ha carries common-land tags. CHIRPS gives a mean annual rainfall of 713 mm for 2006–2025, a 75 %-dependable rainfall of 599 mm and a coefficient of variation of 23 % (373 mm in 2018 to 1 063 mm in 2020); 79 % falls in June–September, over 53 rainy days a year. Reference ET₀ is 1 771 mm/yr, 1 249 mm of it in the dry season. The ranking algorithm found five independent sites, all on open land with slopes of 0.1–1.6° (Table 12, Figure 8).

Table 12. Ranked candidate ponds for Hiware Bazar (soil group C, CHIRPS 2006–2025)

Rank	Score	Location (°N, °E)	Catchment (ha)	Mean runoff (m ³ /yr)	Dep. runoff (m ³ /yr)	Storage (m ³)	Top size (m)
1	0.92	19.07995, 74.59600	137.6	134 368	73 712	36 856	141×94
2	0.89	19.06862, 74.60285	67.3	68 007	37 955	18 978	103×69
3	0.88	19.08247, 74.60760	106.5	105 393	58 190	29 095	126×84
4	0.88	19.06808, 74.59466	78.5	75 657	41 301	20 650	107×72
5	0.84	19.08157, 74.61008	57.7	56 694	31 153	15 576	94×63
Total			447.6	440 119		121 155	

The recommended pond (rank 1) lies on open land at 709 m with a 0.4° slope, at the outlet of a 137.6 ha catchment (perimeter 7.72 km, relief 44.8 m, mean slope 3.3°). The catchment is 77 % open or fallow land, 22 % crops and 1 % built-up (composite CN 85.8), giving a mean runoff of 98 mm/yr — an effective runoff coefficient of 0.14 — or 134 368 m³/yr, and 73 712 m³ in a dependable year. Table 13 gives the design. Dry-season evaporation removes about 44 % of the storage, which is realistic for the Deccan and is why the design prefers depth to area once land is limited. Together the five ponds capture about 28 % of the mean runoff of their catchments, leaving most of the flow to downstream users.

Table 13. Recommended pond, rank 1

Storage capacity	36 856 m³	Water spread area	1.33 ha
Top size (L×W)	141.3 m×94.2 m	Excavation volume	43 604 m ³
Bed size (L×W)	132.3 m×85.2 m	Dry-season evaporation	16 099 m ³ (44 %)
Water depth + freeboard	3.0 m + 0.5 m	Seepage (4 months)	3 384 m ³
Side slope	1.5 : 1	Fills per dependable year	2.0

7.2 Validation Against Reference Data

Each component was checked against independent references.

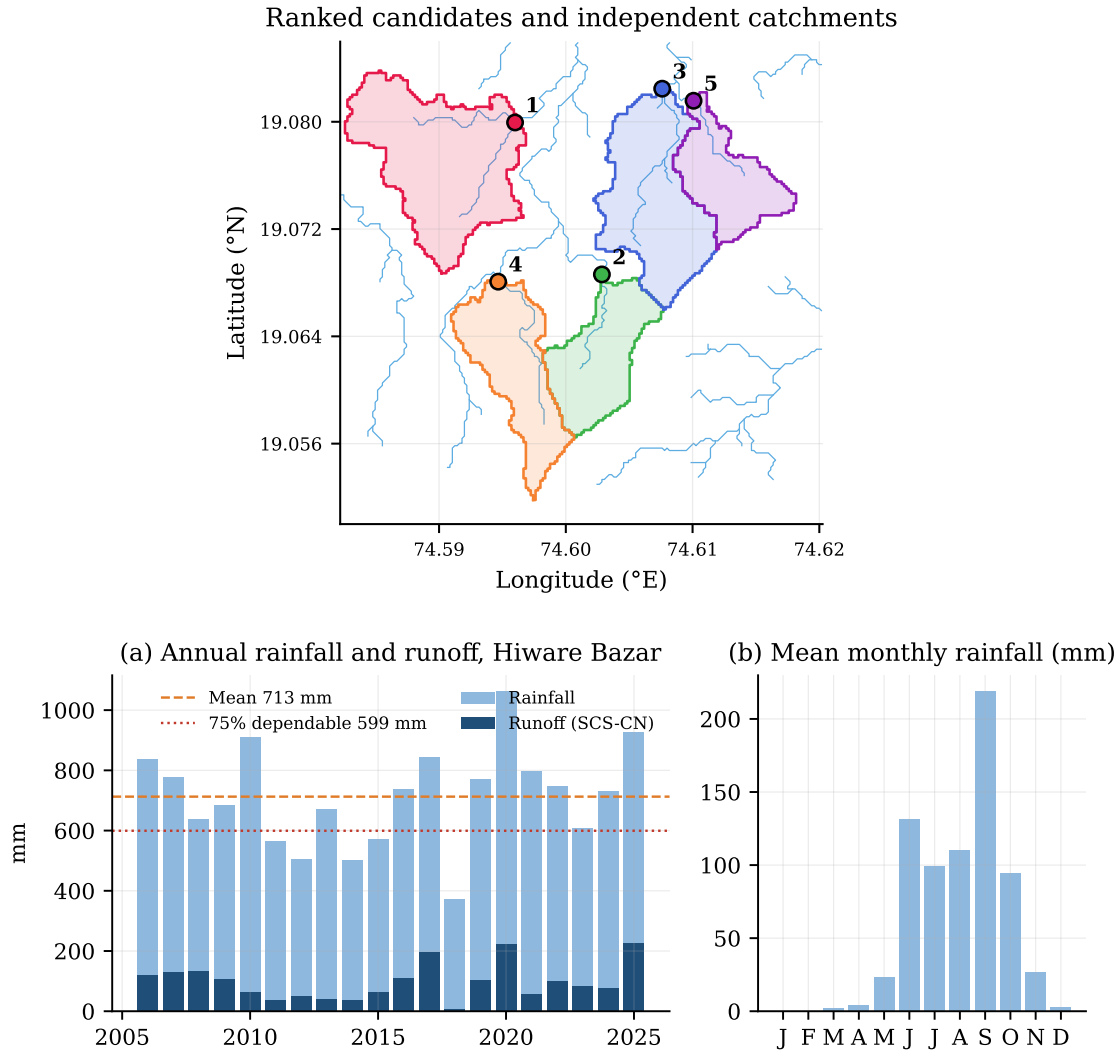


Fig. 8. Example results for Hiware Bazar: (top) the five ranked sites and their independent catchments; (bottom) annual rainfall and simulated runoff depth 2006–2025, and the monthly rainfall climatology.

Rainfall. For six IMD observatories in contrasting climates, each dataset’s 1991–2020 mean annual rainfall was compared with the IMD normal (Table 14, Figure 9). CHIRPS has a mean absolute error of 8.9 %; NASA POWER 26.1 %, with +92 % at Pune, where its ~50 km cells mix the wet windward Ghats with the dry plateau. CHIRPS underestimates the desert (–28 % at Jaisalmer), a known weakness of satellite rainfall in arid regions.

Elevation. Terrain-tile elevations at 44 OpenStreetMap summits with surveyed heights in three regions have a median absolute error of 20 m and a bias of –27 m (Figure 10). Summits are a worst case, since a 30 m DEM averages a sharp peak with its flanks; pond sites lie on gentle valley floors and depend on relative elevation.

Table 14. Mean annual rainfall 1991–2020 (mm) and error relative to the IMD normal

Station	Climate	IMD	CHIRPS	Error	NASA POWER	Error
Ahmednagar	semi-arid, rain shadow	608.1	658.1	+8.2 %	652.0	+7.2 %
Pune	leeward Western Ghats	841.2	897.9	+6.7 %	1616.7	+92.2 %
Jaisalmer	arid	256.9	184.8	−28.1 %	210.1	−18.2 %
Bengaluru	semi-arid plateau	1077.8	1054.0	−2.2 %	858.7	−20.3 %
Chennai	NE-monsoon coast	1408.4	1396.8	−0.8 %	1237.9	−12.1 %
Mumbai	windward coast	2502.3	2684.6	+7.3 %	2341.7	−6.4 %
Mean absolute error				8.9 %		26.1 %

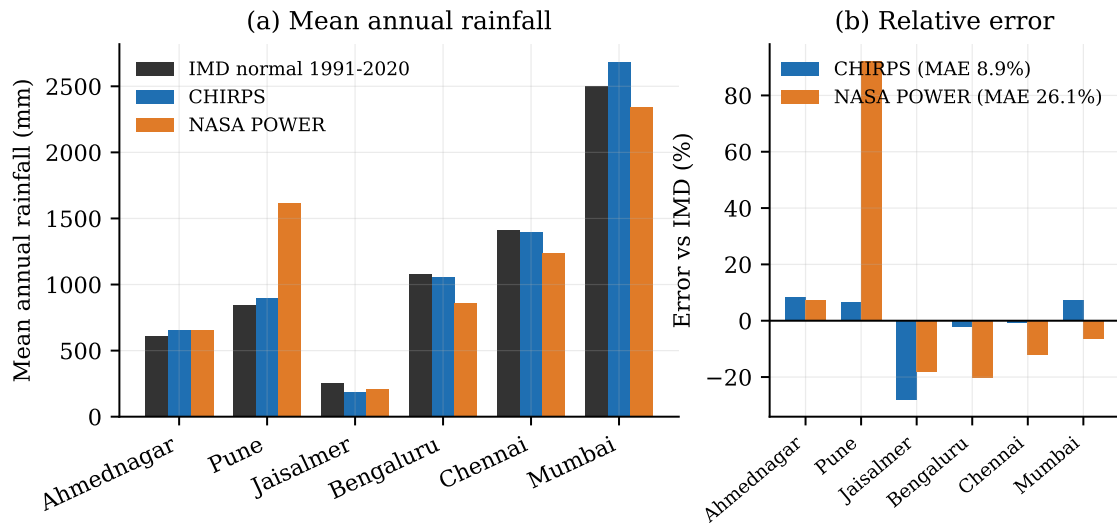


Fig. 9. Rainfall datasets compared with IMD 1991–2020 station normals: (a) mean annual rainfall; (b) relative error.

Land cover. In four landscapes (semi-arid Maharashtra, irrigated Punjab, wet Kerala, arid Rajasthan), 25 of the classifier's 20 m cells each were drawn at random (seed 42), rendered as 100 m zoom-18 image chips without the prediction, and labelled by eye; 13 mixed cells were excluded. Accuracy on the 87 remaining cells is 92 %, $\kappa = 0.85$ (Figure 10): Hiware Bazar 95 %, Ludhiana 100 %, Kuttanad 91 %, Jaipur rural 83 %. Errors are pale sparse grass called vegetation and half-tree cells. The sample contained only two built-up cells and no water cells, so those classes are not validated; OSM is the primary exclusion for both.

Runoff and sensitivity. Runoff could not be compared with measured flow: no gauged village-scale catchments exist in open data. The effective coefficient of 0.14 is plausible for a semi-arid catchment but unverified. The sensitivity analysis (Figure 11) shows an elasticity of about three with respect to rainfall (± 10 % rain gives -30 % / $+31$ % runoff, where the rational method would give ± 10 %), and a factor of 4.7 between soil groups A and D. Rainfall and soil group therefore dominate the uncertainty in pond sizing — which is why the rainfall source was validated and made correctable, and the soil group is exposed with plain descriptions.

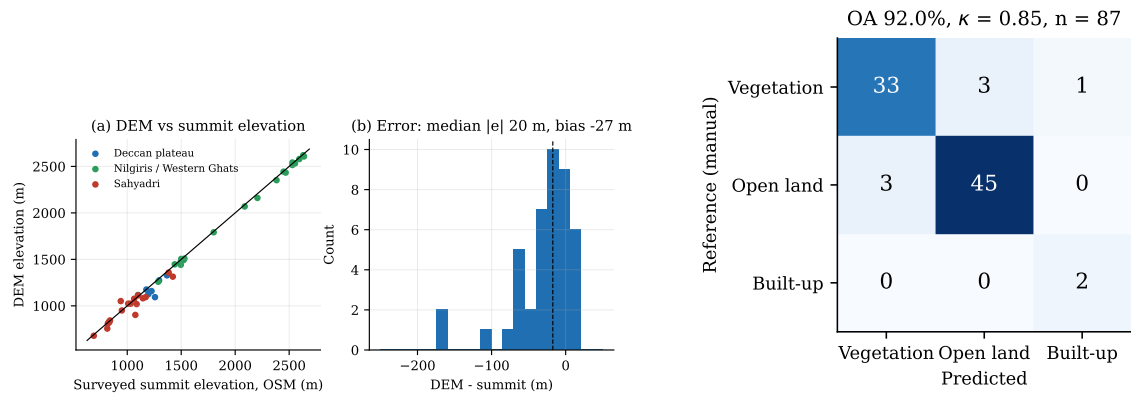


Fig. 10. (left) DEM against surveyed summit elevations; (right) land-cover confusion matrix on 87 blind-labelled cells.

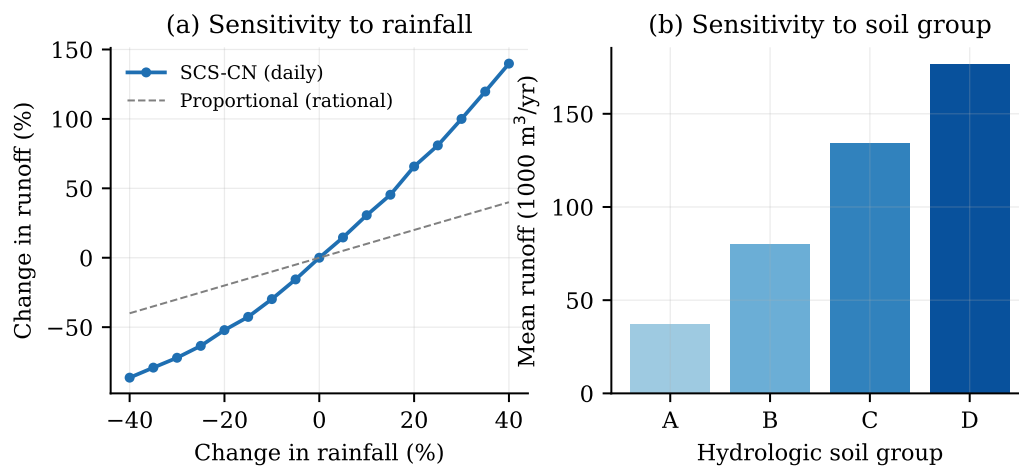


Fig. 11. Sensitivity of mean annual runoff at Hiware Bazar (rank 1): (a) to a uniform scaling of daily rainfall; (b) to hydrologic soil group.

7.3 Testing Strategy

Testing is layered (Table 15). Unit tests pin formulas and the hydrology core to hand calculations and exact analytic answers – most notably a synthetic V-valley whose catchment is known cell for cell. API tests start the real application in-process and check routing, validation and error shapes without network access. System tests drive the running server over HTTP with real data, and load tests drive it concurrently.

7.4 Defects Found by System Testing

All unit tests passed from the start, yet system testing, load testing and the lab deployment found six defects (Table 16). Each was fixed, and five now have regression tests.

Table 15. Test levels

Level	Tests	What it checks
Unit: formulas	19	SCS runoff (hand-calculated 50.54 mm case and four more), AMC conversion, runoff volume, frustum volume, width solver, pond rules (incl. no-fit), FAO-56 radiation example, rainfall statistics
Unit: hydrology	5	No sinks after filling; analytic V-valley catchment; polygon area = cell count; bowl drainage; edge-inflow flag
Unit: siting	4	No site in the sea, even in the terrain-only fallback; sea mask keeps an inland polder
Unit: OSM client	2	Second query served from cache; failure starts the fail-fast cool-down
API	5	App imports and starts; front end and OpenAPI served; error shapes for 422, 400, 404
System (end to end)	10	Health, geocoding, rainfall, cached and new villages, manual pin, contour upload, ocean and bad-input rejection, history — on the lab server
Load	5	1–16 concurrent analyses; 50 concurrent health checks
Validation	3 studies	Rainfall (6 stations), elevation (44 summits), land cover (100 cells)

Table 16. Defects found after the unit tests passed, and their fixes

#	Found by	Symptom	Cause and fix	Regression test
1	End-to-end probe of unusual inputs	A pond 4 288 m below sea level in the Pacific; 3 of 5 coastal Konkan sites offshore	The RGB water rule misses teal, turbid water, and Esri serves grey placeholders over the ocean. Fix: DEM-based sea mask (connected low areas with sea-surface or bathymetry seeds), 422 for mostly-sea areas, placeholder detection	4 tests
2	Load test, 4 concurrent analyses	45 s per request instead of 1.3 s; all results silently lacked OSM exclusions	Overpass admits ~2 queries per IP; OSM was not cached. Fix: 7-day cache, serialised calls, 2-minute circuit breaker	2 tests
3	Lab deployment	Analyses hung for 70 s or more on “finding the terrain”	The campus firewall silently drops Overpass connections (SYN-SENT); each of two servers waited the full 35 s read timeout. Fix: 5 s connect timeout	— (network)
4	Lab deployment	“No elevation source available” for new villages	DNS failed 2 times in 10; one failed tile of ~30 aborted the analysis and the 3 retries had no pause. Fix: 5 attempts with back-off	— (network)
5	Lab deployment (Hyderabad)	HTTP 500, ZeroDivisionError	In a dense city a land patch can be smaller than the smallest pond; storage was capped to 0 and divided by. Fix: report “no pond fits here”	1 test
6	Code comparison between laptop and server	App would not start from the laptop copy	An accidental edit left a syntax error in <code>main.py</code> ; no unit test imported it. Fix: restored file; added API tests that import and start the app	5 tests

The lesson is that a geospatial web application’s hardest failures are at its boundaries — unusual places on Earth, other people’s servers, and real networks — which only system-level testing exercises.

7.5 Performance

Laptop timings were measured through the HTTP API on an Apple M4 (10 cores), 2–4 October 2026; server timings on the lab machine (Ubuntu 24.04, 120 cores) on 5 October (Table 17). Figure 12 breaks a cached analysis into stages.

Table 17. Measured response times

Operation	Laptop	Lab server
Village analysis, first run (20 years of CHIRPS downloaded)	44.2 s	59–317 s*
Village analysis, cached (Hiware Bazar)	1.3 s	3.3 s
Contour-map analysis (sample_contours.kml), first run	64.6 s	171 s*
GET /api/rainfall, cached	8 ms	—
GET /api/geocode, cached	1 ms	—
GET /health, 50 concurrent	median 2.4 ms, p95 3.3 ms	
Test suite (35 tests)	0.4 s	4.0 s

*Dominated by the unreliable campus network: timeouts on unreachable services before falling back.

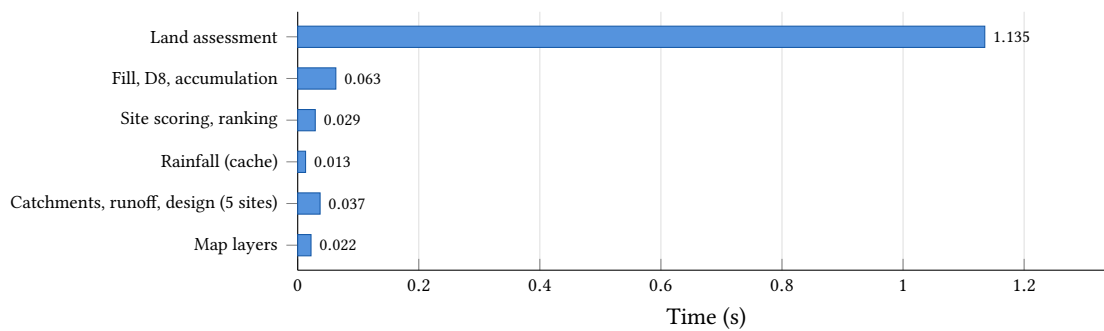


Fig. 12. Stage times of a cached village analysis (total 1.30 s). Land assessment — downloading and classifying imagery — dominates; the hydrology on 40 000 cells takes 63 ms.

A first analysis is dominated by the rainfall download (about 35 s for two parallel ten-year CHIRPS requests). The computation itself is fast: D8 is vectorised and accumulation is a single sorted pass.

7.6 Behaviour Under Load

Concurrent cached analyses of the same village were sent to one server instance in three versions of the code (Figure 13, Table 18). In the baseline, Overpass's per-IP limit was exceeded: requests beyond the first two waited about 44 s and lost their OpenStreetMap data. Caching and serialising Overpass calls alone made the failure case worse: Overpass had meanwhile started refusing our network (a temporary block, most likely triggered by the load test), every query failed, nothing was cached, and requests queued behind each other's timeouts — up to 293 s. With the circuit breaker, a batch of four shared a single 36 s timeout and the next eight requests completed in 1.3 s each, with the missing OSM data reported in their warnings. Because NumPy and OpenCV release Python's global interpreter lock, eight concurrent analyses took no longer than one: CPU is not the bottleneck, external APIs are. The successful-cache path is covered by unit tests but could not be load-tested live while Overpass blocked our network.

8 DISCUSSION AND LIMITATIONS

What worked well. Building everything on one in-memory DEM made the two input modes share all downstream code. Validating data sources early changed the design for the better (the switch to CHIRPS). Returning methods,

Table 18. Concurrent cached analyses: latency (s) by code version

Code version	1 request	4 concurrent (median / max)	8 concurrent (median / max)
Baseline (no OSM cache)	1.37	45.15 / 45.31	2.27 / 44.39
+ cache and serialised calls (Overpass down)	36.26	71.99 / 144.00	144.07 / 293.62
+ circuit breaker (Overpass down)	—	36.50 / 36.53	1.32 / 1.35

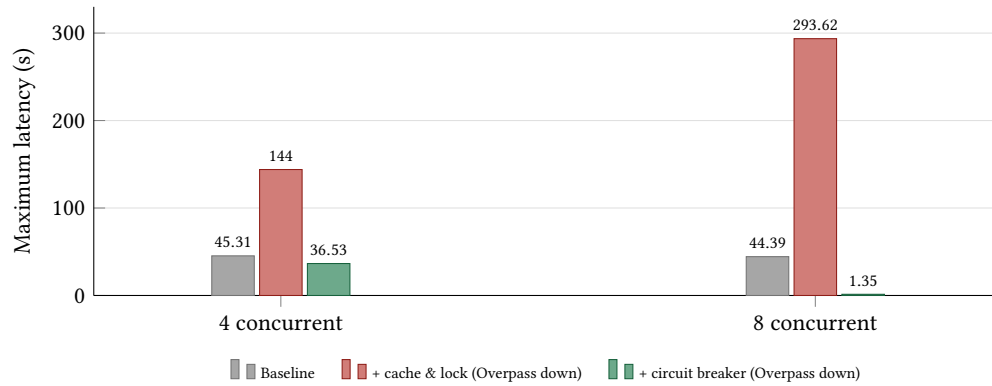


Fig. 13. Worst-case latency of concurrent cached analyses in three code versions. The circuit breaker turns one timeout per request into one timeout per outage.

timings and warnings with every result made failures visible instead of silent. Testing beyond unit tests — odd locations, concurrency, a real deployment — found every serious defect.

What did not. The RGB water rule (blue brighter than red and green) does not recognise the turbid, green-tinted water typical of Indian coasts and reservoirs; sea is now excluded using elevation, but inland water still relies on OpenStreetMap. Dependence on live public services makes results drift as those services change, and some rate-limit or block aggressively; on the campus network, analyses often run without OpenStreetMap or imagery, which degrades siting (a contour analysis returned a 409 ha catchment instead of 5.4 ha because a river was no longer excluded). Results always state this in their warnings, but a planner can still overlook it.

Data quality and assumptions. The ~30 m DEM cannot resolve field bunds, drains or road embankments, which can redirect flow on the ground. CHIRPS underestimates deserts (−28 % at Jaisalmer). Soil group is a user input and, after rainfall, the most influential parameter. TR-55 curve numbers were developed for US catchments; their use in India is standard in watershed manuals but not locally calibrated. Land ownership is inferred, never certain, and groundwater depth and rock are ignored when choosing depth. All recommendations therefore need field verification.

Future work. Sentinel-2 bands (NDWI, NDBI) for water and built-up detection; ISRIC SoilGrids for automatic soil groups; IMD gridded rainfall as a source; cost per cubic metre of storage to rank candidates; and a GitHub Actions pipeline running the test suite on every push.

9 AI TOOL USAGE DECLARATION

[TODO: List every AI tool you used while developing the project (e.g. ChatGPT, Claude, Gemini, GitHub Copilot) and what you used each for. The paragraph below covers only the final review, deployment and reporting sessions.]

Claude Code (Anthropic, model Claude Opus 5.5) was used in the final stage of the project for: reviewing the code and running the application end to end; finding and fixing the six defects in Table 16 and writing their regression tests; load testing and measuring the performance figures in Section 7.5; deploying the application on the college lab server; publishing the repository to GitHub after scanning its history for secrets; updating the README, validation notes and the main report; and writing this report in the ACM template from the existing technical report and the measurements above. **[TODO: Confirm honestly: “All AI-assisted code and text was reviewed, understood and, where needed, modified by me before submission.”]**

REFERENCES

- [1] Ministry of Rural Development, Government of India. 2022. *Mission Amrit Sarovar: Guidelines*.
- [2] R. Barnes, C. Lehman, and D. Mulla. 2014. Priority-flood: An optimal depression-filling and watershed-labeling algorithm for digital elevation models. *Computers & Geosciences* 62, 117–127.
- [3] J. F. O’Callaghan and D. M. Mark. 1984. The extraction of drainage networks from digital elevation data. *Computer Vision, Graphics, and Image Processing* 28, 3, 323–344.
- [4] N. Otsu. 1979. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics* 9, 1, 62–66.
- [5] C. Funk et al. 2015. The climate hazards infrared precipitation with stations — a new environmental record for monitoring extremes. *Scientific Data* 2, 150066.
- [6] H. Hersbach et al. 2020. The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society* 146, 730, 1999–2049.
- [7] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith. 1998. *Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements*. FAO Irrigation and Drainage Paper 56, Rome.
- [8] USDA Soil Conservation Service. 1986. *Urban Hydrology for Small Watersheds*. Technical Release 55, 2nd ed.
- [9] R. H. Hawkins, A. T. Hjelmfelt, and A. W. Zevenbergen. 1985. Runoff probability, storm depth, and curve numbers. *Journal of Irrigation and Drainage Engineering* 111, 4, 330–340.

A SOURCE CODE AND REPOSITORY

Repository (public): <https://github.com/amandeeip01/village-pond-planner>

Video explanation and demo: <https://www.youtube.com/watch?v=StI46Pxp3kM>

app/ Back end: main.py (routes), schemas.py (API models), pipeline.py (orchestration), elevation.py, tiles.py, terrain.py, kml_parser.py (terrain), hydrology.py, landcover.py, pond.py, rainfall.py, runoff.py, design.py, layers.py (analysis), geocode.py, db.py (I/O).

app/static/ Front end: index.html, app.css, app.js.

tests/ Unit and API tests (pytest).

validation/ Scripts and results for the rainfall, elevation and land-cover validation.

report/ Reports, figures, the saved case-study analysis and the script that regenerates the figures.

Root README.md (usage, API, methods), VALIDATION.md, requirements.txt, requirements-dev.txt, Dockerfile, render.yaml and sample_contours.kml.

B RUNNING THE SYSTEM

```
python3 -m venv .venv
.venv/bin/pip install -r requirements-dev.txt
```

```
1197 .venv/bin/python -m pytest -q tests # 35 passed
1198 .venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000
1199 # app: http://localhost:8000 API docs: http://localhost:8000/docs
1200
1201 # Docker
1202 docker build -t pond-planner . && docker run -p 8000:8000 -v pond-data:/app/data pond-planner
1203
1204 # Lab server (no Docker): start | stop | restart | status | logs
1205 ~/pond-planner/pond.sh start
1206 ssh -N -L 8080:localhost:8000 sys4 # then open http://localhost:8080
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
```